FreelanceFlow

A freelancer workspace from client management to time-based invoicing

Prepared by: Atharva
Prepared for: Project lead review
Report date: 20 October 2026
Scope: Four development milestones and deployed demonstration

Live application
https://freelanceflow-steel.vercel.app/

Source repository
https://github.com/Atharva16bit/FreelanceFlow

Delivery assessment

The core demonstration workflow is implemented and deployed. Development included manual API tests, browser checks, and successful frontend lint/build runs. Production readiness, automated regression coverage, and a comprehensive security assessment remain outside the completed scope.

Problem, objective, and delivered scope

Freelancers often maintain client details, task lists, work logs, and invoices in separate tools. FreelanceFlow connects those records so that completed work can become an invoice without manually re-entering its duration and hourly rate.

Objective

Build a full-stack application where each freelancer can manage their own business records, track work, generate invoices, and review outstanding and collected amounts.

Area	Delivered outcome
Accounts	Registration, login, logout, and PostgreSQL-backed sessions.
Work management	Client, project, and task screens with create/read/update/delete workflows.
Time tracking	One running timer per user, refresh recovery, manual entries, and finished history.
Billing	Unbilled-entry selection, transactional invoice creation, payment status, and Pro PDF downloads.
Visibility	Dashboard totals, upcoming tasks, and six calendar months of financial data.
Delivery	GitHub source, Vercel deployment, Neon database, schema export, and repeatable demo setup.

Scope boundary

This is a deployed project demonstration. A paid invoice means the user manually recorded payment; the application does not collect money. Free and Pro entitlements exist, but a subscription purchase workflow is not implemented.

Evidence basis

This report uses the development outputs and test responses recorded in the project conversation, the project setup record, and the nine-page screenshot PDF supplied by the developer. Screenshot evidence and earlier test evidence are distinguished. The repository was not independently audited during report preparation.

Architecture and request handling

Component	Responsibility
React + Vite frontend	Forms, lists, dashboard, timer display, and same-origin API calls.
Express backend	Input validation, session authentication, ownership checks, business rules, and PDF generation.
PostgreSQL	Related business records, sessions, monetary values, constraints, and transaction boundaries.
Vercel	Serves the frontend build and routes API requests to the Express entry point.
Neon	Hosts the deployed PostgreSQL database.

A typical request

The browser calls an /api endpoint with its session cookie. Express loads the stored session, validates the request with Zod, and checks ownership in its SQL query. PostgreSQL returns the authorized rows. Express sends JSON, and React updates the displayed state. PDF endpoints return a downloadable document.

Local and hosted execution

Locally, the frontend runs on localhost:5173 and proxies /api to the backend at 127.0.0.1:5000. In the hosted deployment, the frontend and API share an origin. The root api/index.js exports the Express application for Vercel.

Key implementation choices

Persistent sessions: connect-pg-simple stores sessions in PostgreSQL so they are not tied only to one process.
Password hashing: bcrypt hashes passwords; plaintext passwords are not returned by account APIs.
Validation: Zod validates request shapes and limits; database constraints provide an additional integrity layer.
Query parameters: SQL values are passed as parameters rather than inserted into SQL strings.
Money: PostgreSQL NUMERIC and database rounding are used for invoice amounts.

Source organization

Backend modules separate authentication, clients, projects, tasks, time entries, invoices, and dashboard queries. Invoice creation, status, reads, and PDF rendering have separate files. React components follow the same business modules. Setup instructions belong in README.md; development history belongs in docs/progress.md.

Eight tables and their relationships

Table	Purpose and primary relationship
users	Account details, unique email, password hash, and plan.
user_sessions	Session identifier, serialized session data, and expiration.
clients	Client contact details and hourly rate; belongs to a user.
projects	Name, status, description, and budget; belongs to a client.
tasks	Title, status, and due date; belongs to a project.
time_entries	User/project, source, start, and end timestamps.
invoices	Owner/client, saved client details, dates, status, and payment timestamp.
invoice_items	Invoice and unique time-entry reference; saved work details and rate; generated duration and amount.

Relationships

One user can own many clients; one client can have many projects; one project can have many tasks and time entries. An invoice contains one or more invoice items. Each selected time entry can appear in only one invoice item. Ownership of projects and tasks is resolved through the related client.

Integrity safeguards

Foreign keys preserve references. Linked clients and projects cannot be deleted while dependent rows remain. A partial unique index permits only one unfinished timer per user. Checks enforce valid statuses, non-negative monetary values, and end times after start times. Invoice status and paid_at must agree.

Invoice snapshot example

A two-hour entry billed at INR 875 per hour produces an item amount of INR 1,750.00. The invoice saves that rate and work description. Changing the client rate later does not rewrite the existing invoice. Project estimates, however, use the current client rate and can change.

Ownership checks are application query rules. This report does not claim PostgreSQL row-level security is implemented.

Four planned milestones

Milestone	Implementation and verification
Week 1: Foundation and CRM	Verified tooling and database connectivity; initialized Git; built registration and session authentication; added client, project, and task APIs/screens. Checked invalid credentials, validation, ownership isolation, the two-client Free limit, and dependent-record deletion conflicts.
Week 2: Time and budgets	Added time_entries and one-running-timer constraint. Implemented start/stop/recovery, manual entries, project filtering, and history pagination. Added current-rate project work-value and remaining-budget estimates. Checked duplicate timers, wrong-owner requests, and invalid manual dates.
Week 3: Invoicing	Added invoice headers/items, unbilled selection, transactional creation, duplicate-billing protection, saved rates, status changes, and PDF downloads. Checked ownership, repeated paid updates, and Free/Pro PDF permissions.
Week 4: Visibility and hosting	Added dashboard cards, upcoming tasks, six-month financial overview, repeatable Pro demo setup, startup/build scripts, hosted database support, and root Vercel deployment configuration. Completed manual live workflow checks.

Issues resolved during development

Session middleware order: authentication routes initially ran before sessions existed. Registering session middleware first resolved the missing regenerate method.
Unreachable route registration: handlers placed after return router were moved before the return.
SQL parameter inference: inconsistent parameter types were corrected with appropriate SQL typing.
Hosted build: frontend development dependencies had to be installed for Vite to be available.
Hosted routing: deployment moved to a root configuration with separate static output and API routing.

The week labels describe delivery milestones, not evidence of four full calendar weeks of work. Development records shown in this conversation span 16-20 October 2026.

Recorded results and their limits

Scenario	Recorded result
Authentication	Valid login succeeded; duplicate email 409; invalid/unknown credentials 401; missing session 401; logout removed access.
Record isolation	A second account received empty lists and 404 responses for another owner's clients/projects/tasks/invoices.
Plan and deletion rules	Third Free client rejected with 403; linked client/project deletion rejected with 409.
Timer	Start/stop succeeded; second running timer 409; repeat or wrong-owner stop 404; refresh recovery confirmed.
Manual entries	Valid two-hour entry saved; end-before-start and future-time inputs rejected with 400.
Invoice safeguards	Invoice total INR 1,750.00 verified for the sample; repeat billing rejected with 409; billed entry removed from eligible selection.
Status and PDF	Repeated paid update preserved paid_at; unpaid reset it; wrong-owner invoice/PDF access 404; Free PDF download 403.
Dashboard	Owner summary returned expected data; second account's totals and lists were empty/zero.
Frontend checks	npm run lint and npm run build completed successfully in the supplied terminal outputs.

Live verification

The developer confirmed sign-in, session persistence after refresh, dashboard display, PDF downloads, and timer refresh/stop behavior on the hosted application. The supplied screenshots document displayed dashboard, CRM, time-log, and invoice interfaces. They do not independently establish every backend edge case.

Checks still needed

No automated regression suite, load test, full security audit, or database restore rehearsal is evidenced. Browser screenshots alone do not verify concurrent invoice creation, every timezone boundary, long multi-page PDF layout, or all malformed inputs. These should be included in a future test plan.

Reproducible setup and hosting

Local setup sequence

1. Install a compatible Node.js runtime and PostgreSQL 18 tools.
2. Clone the repository and create a database owned by the application role.
3. Import backend/db/schema.sql into the empty database.
4. Configure backend/.env with database settings and a random SESSION_SECRET of at least 32 characters.
5. Install backend and frontend dependencies using npm ci.
6. Run npm start in backend and npm run dev in frontend.
7. Open localhost:5173; check the backend /api/health endpoint.

Vercel setting	Current configuration
Root directory / preset	Repository root (./) / Other
Install	npm ci --prefix backend && npm ci --prefix frontend --include=dev
Build	npm run build --prefix frontend
Output	frontend/dist
API	api/index.js; /api/:path* rewrite to /api/index
Environment	NODE_ENV=production; DATABASE_URL; SESSION_SECRET

Database and demo setup

The Neon schema was imported separately with psql. The schema-only export creates tables and constraints, not account data. seedDemo.js creates the dedicated Pro demo account, three clients, three projects, three tasks, five paid invoices, one unpaid invoice, and one unbilled entry. Repeating the seed leaves an existing demo account unchanged.

Operational notes

Database credentials and session secrets must remain outside Git and screenshots. The hosted database connection requires SSL settings. Updating hosted environment variables requires a new deployment. The schema export supports a fresh setup; it is not a migration framework or a data backup.

The live demo is editable. Its values may change after screenshots were captured. Do not use demonstration records as accounting evidence.

Limitations and priorities

Priority	Recommended work and reason
Before broader use	Rotate any database credentials previously exposed in development messages; confirm environment isolation and least-privilege database access. Completion of rotation is not evidenced here.
Security hardening	Review CSRF/origin protections, cookie/proxy behavior, distributed rate limiting, request validation, and error handling under the actual serverless deployment.
Regression coverage	Automate authentication, ownership, timer uniqueness, invoice creation races, rounding, and payment-state tests against an isolated test database.
Database operations	Introduce versioned migrations, a backup/restore procedure, and query/index monitoring. Validate a restore rather than relying only on schema export.
Account lifecycle	Implement password reset, email verification, and a documented demo-data reset policy.
Product completeness	Add time-entry corrections with billing safeguards, scalable list pagination, clearer refresh synchronization, and a real entitlement upgrade workflow if required.
Business integrations	Treat payment processing, taxes, subscription billing, and accounting integrations as separate future scope.

Project lead handover

The delivered application demonstrates the complete client-to-project-to-time-to-invoice workflow. Its strongest integrity features are ownership-scoped queries, persistent sessions, transactional invoicing, unique billing references, and saved invoice values. The next phase should focus on repeatable tests and deployment hardening before adding more features.

Suggested review demonstration

Sign in to the demo account; inspect the dashboard; show a client and project; start a timer and refresh; stop it and inspect history; select unbilled work; create an invoice; view its saved amount and download the PDF; change payment status and refresh the dashboard. Use clearly labelled test records in the shared demo.

Dashboard and financial overview

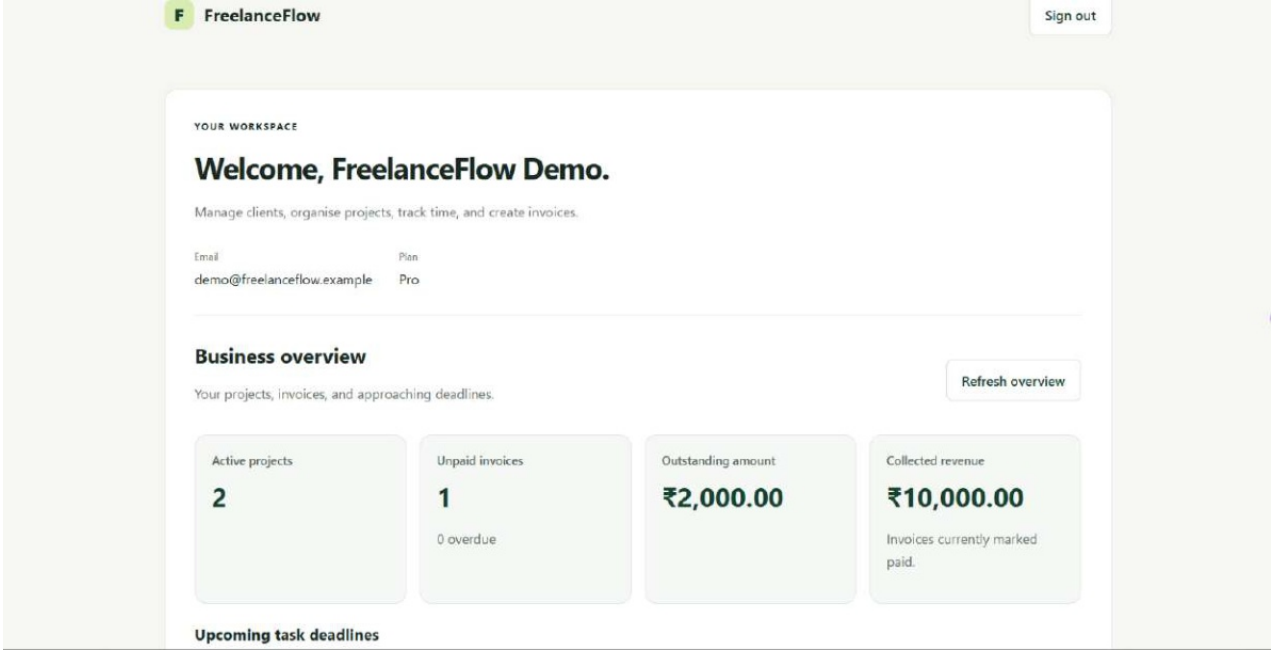


Figure 1. Supplied dashboard screenshot: two active projects, one pending invoice, INR 2,000 outstanding, and INR 10,000 collected.

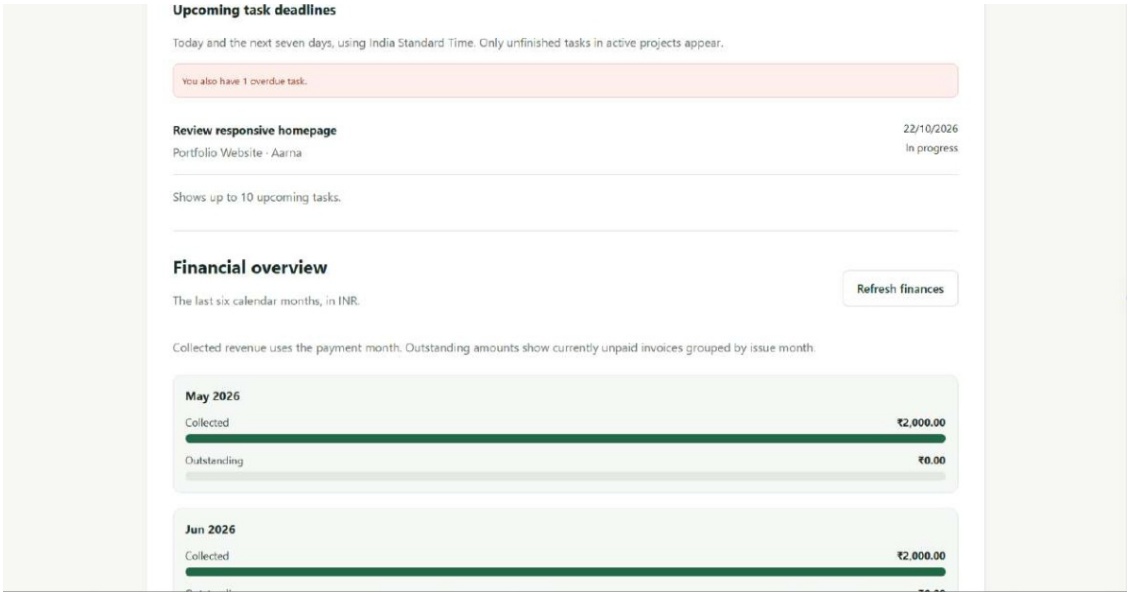


Figure 2. Upcoming tasks and the beginning of the six-month financial overview.

The overdue-task message is visible in the supplied capture. Snapshot values describe this demo workspace at capture time.

Financial chart and client workspace



Figure 3. Financial overview showing monthly collected and outstanding amounts.

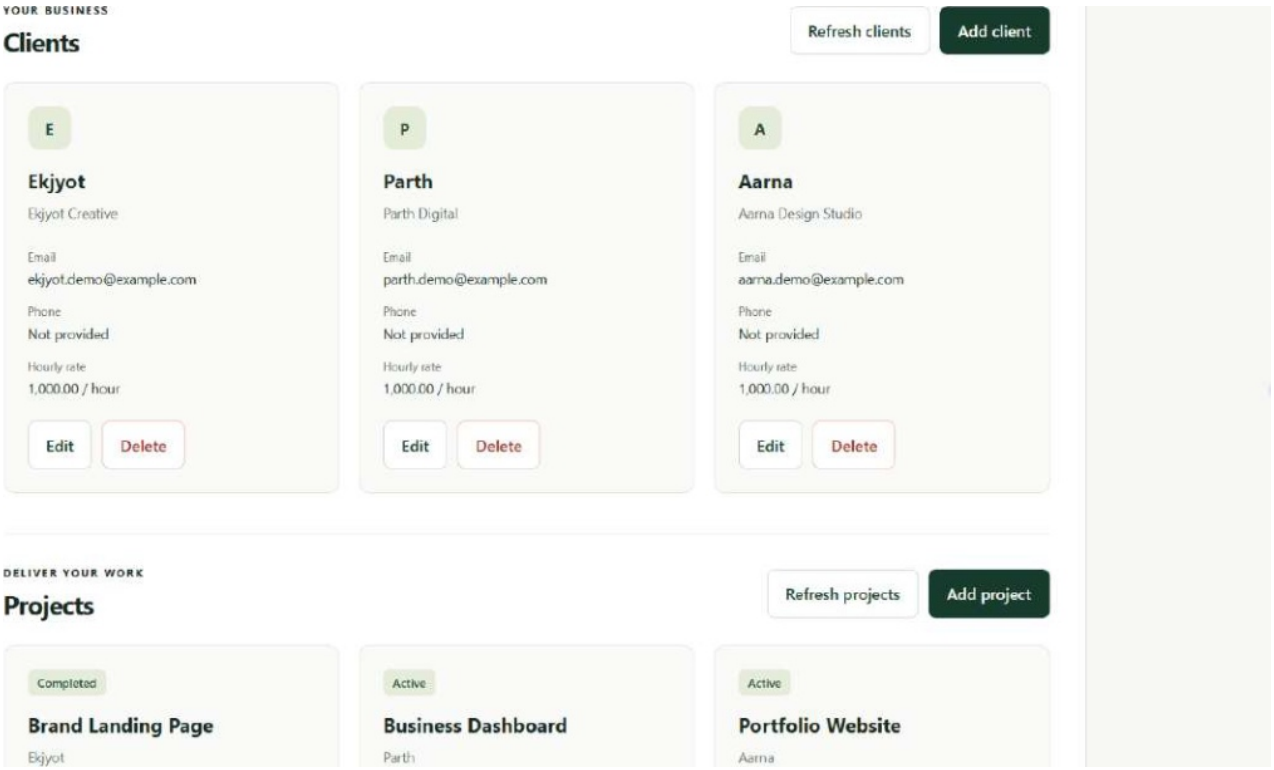


Figure 4. Client cards and the beginning of the project section.

Projects and tasks

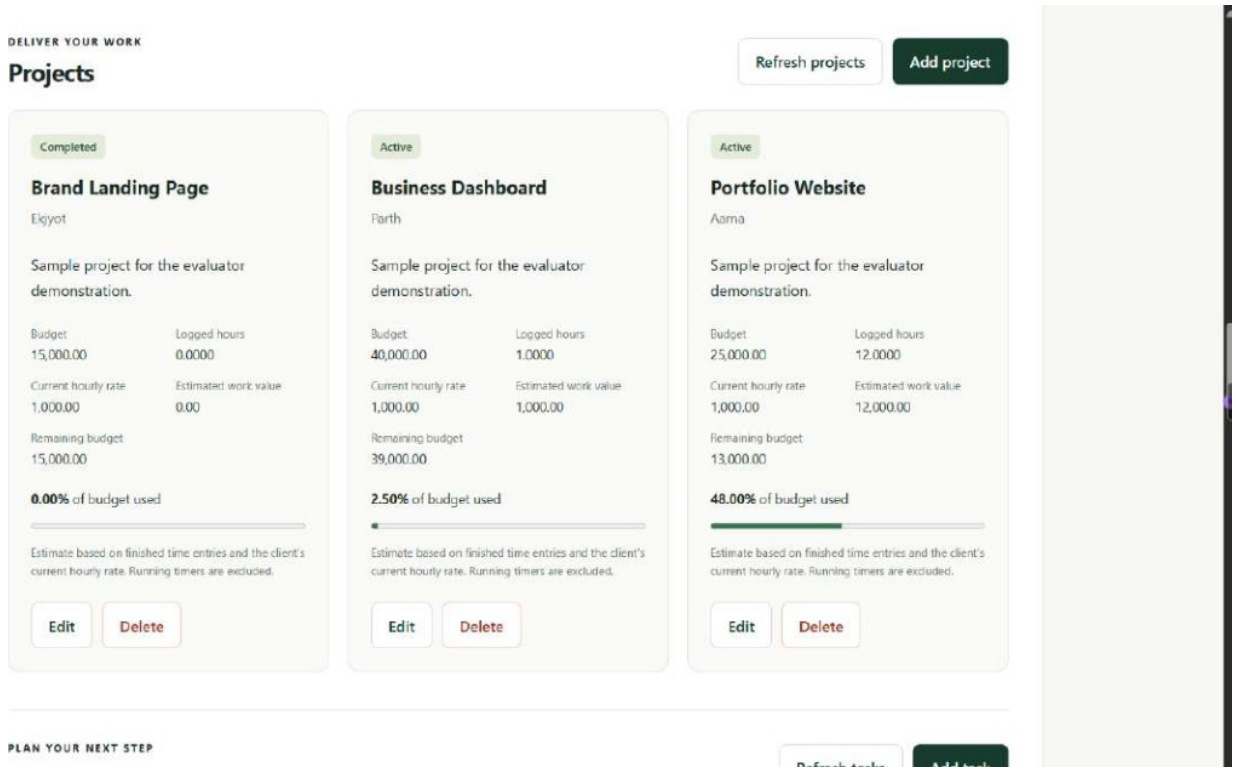


Figure 5. Project cards show status, budget, and logged-work estimates.

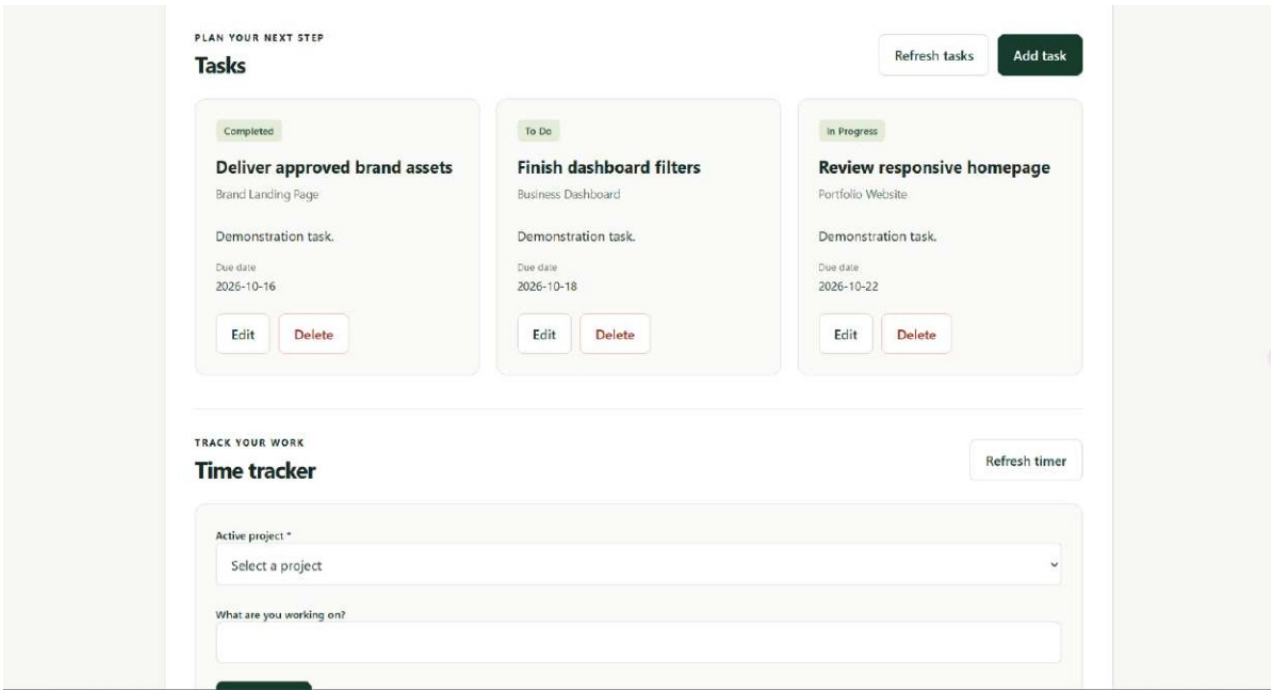


Figure 6. Task cards show work status and due dates; the timer form begins below.

Time history and invoice creation

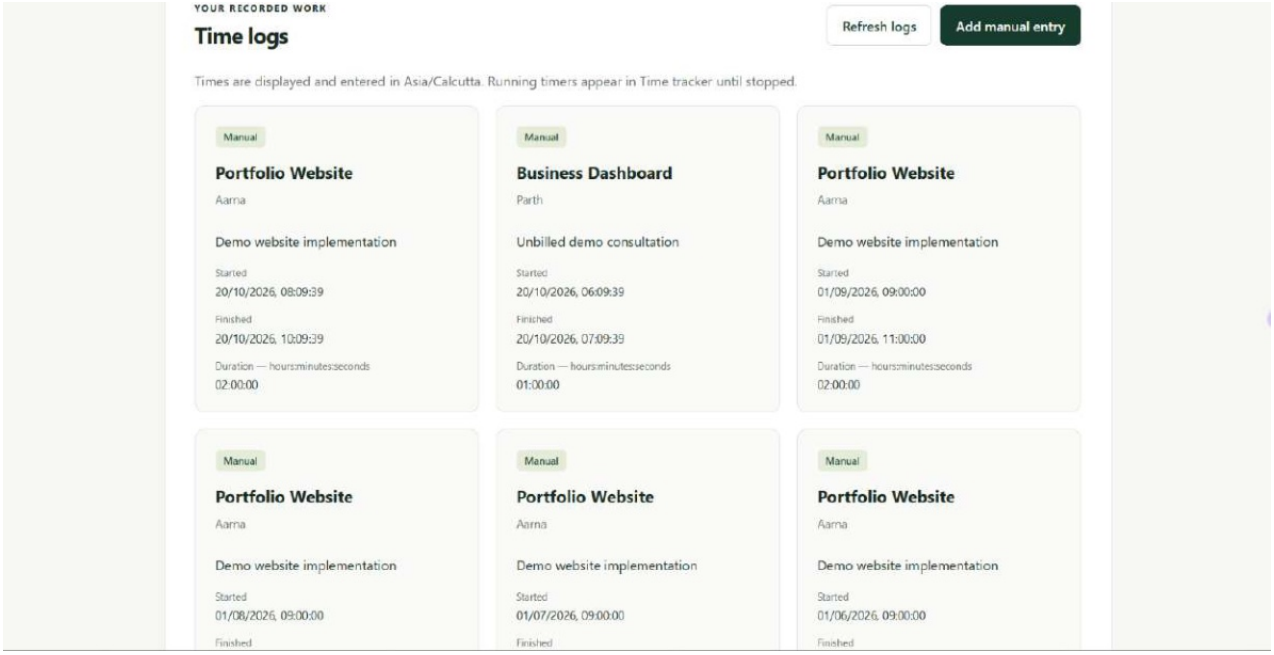


Figure 7. Finished time entries displayed in Time logs.

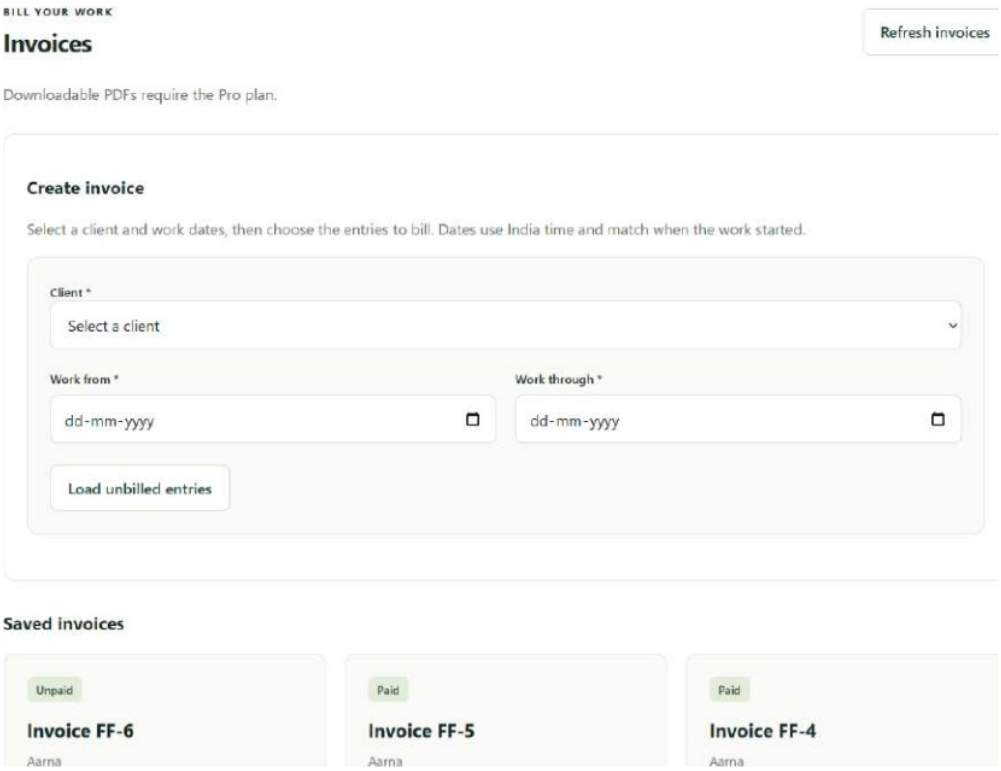


Figure 8. Invoice creation form and the start of the saved invoice list.

Saved invoices and evidence register

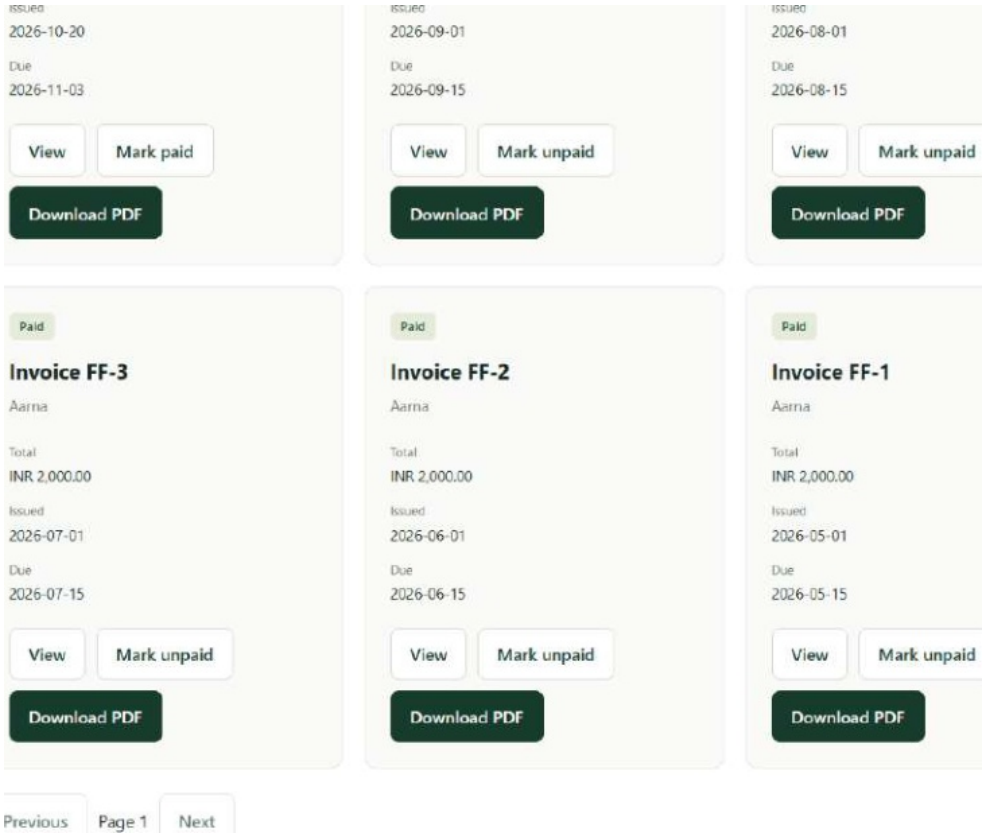


Figure 9. Saved invoice cards include view, status, and PDF-download controls.

Evidence register

Screenshot source: Screenshot 2026-10-20 142714.pdf, nine pages, supplied by the developer. Blank PDF page margins were removed for report layout; screenshot contents were not rewritten.
Additional source: Project conversation terminal outputs and developer confirmations, including successful lint/build results and live timer verification.
Documentation: README.md setup/delivery guide and docs/progress.md development record.
Source code: GitHub repository linked on the cover. No independent line-by-line source review was performed for this report.

Missing visual evidence

The supplied PDF does not include a running-timer screenshot, opened invoice-item detail view, or the rendered downloaded invoice PDF. Their workflows are supported by earlier developer confirmations and API outputs, not by those screenshot figures. No replacement screenshots were fabricated.